Contents

Axona Programmer Guide

A practical guide to building a decentralized pub/sub application on the Axona protocol. After reading this, you should be able to ship a working chat, feed, or notification system without reading any other Axona code.

The guide assumes you already know JavaScript + Node + a browser. It does not assume any DHT / WebRTC / cryptography background — concepts are introduced where they're needed.

- **Protocol kernel**: @axona/protocol (v2.32.0)
- **Browser SDK**: @axona-net/axona-peer (v3.27.0)
- **WebSocket bridge**: @axona-net/axona-bridge (v2.14.0)
- **Live network**: `wss://bridge.axona.net`
- **Security model**: see §3.5 below and the security changelog — the v2 line adds an authenticated-identity handshake, channel binding, a pub/sub trust boundary, and verified routing admission.

Table of contents

15. API reference cheat sheet

---

1. What is Axona?

Axona is a peer-to-peer mesh protocol where every participant is an equal node — there is no central server. Three things make it useful:

1. **Geographic routing**: every node's ID encodes the geographic cell it lives in (an 8-bit Google S2 prefix). Messages and lookups are XOR-routed in a way that strongly favors same-region neighbors, so a us-east publisher's traffic naturally clusters on us-east peers.
2. **Signed pub/sub**: every published message is signed with the publisher's Ed25519 key. Subscribers verify cryptographically that the envelope came from whoever it claims to.
3. **Replicated cached delivery**: each topic has a set of "axons" — the K closest peers to the topic ID — that hold a bounded ring of recent messages. Late subscribers can replay history; live subscribers tail in real time.

The protocol kernel (`@axona/protocol`) is environment-neutral pure JS. It runs in browsers, Node, and in-process simulation. Three deployed components ship on top of it:

Component	What it is	Where it runs
`axona-peer`	Browser SDK + reference client	Static-served HTML/JS — any web host (axona.net uses GitHub Pages)
`axona-bridge`	WebSocket signaling broker + universal-hub peer	A small Node process behind nginx + TLS
`dht-sim`	Visualization + analysis simulator	Node, locally

The bridge is a signaling broker for WebRTC and a universal-fallback peer in the mesh — it's NOT a centralized message queue. Most messages flow directly between browsers over WebRTC data channels; the bridge is a WebSocket fallback when NAT prevents direct connection, and a hub that helps mesh discovery on cold-start. The protocol does not depend on the bridge for correctness.

Address space (important)

Every node ID and every topic ID is 264 bits, encoded as 66 lowercase hex characters. The layout is the same for both:

```
S2 cell prefix          256-bit pubkey-derived / sha256 hash
8 bits (top 2 hex)      64 hex chars
```

For node IDs, the bottom 256 bits are `sha256(pubkey)`. For topic IDs, they are `sha256(<addressing prefix>:<topic name>)` — see §5.

Because peer IDs and topic IDs share the same address space, XOR distance between a peer and a topic is meaningful: peers in the same S2 cell as the topic are XOR-close to it, and the routing layer uses that to keep locality.

---

2. Quick start: 15 minutes to a pub/sub roundtrip

The fastest path is to **use the public bridge** at `wss://bridge.axona.net` and write a tiny browser peer. Two HTML files (one publisher, one subscriber), no build step. We'll use the kernel directly via `<script type="module">` and skip the WebRTC mesh by going through the bridge only.

2.1 Set up a project

```
mkdir my-axona-app && cd my-axona-app
npm init -y
npm install @axona/protocol@github:axona-net/axona-protocol#v2.32.0
```

You now have `node_modules/@axona/protocol/src/` with the full kernel.

2.2 A minimal node

Create `node.js`:

```
// node.js - minimal Axona node connected to a bridge over a WebSocket.
import {
  AxonaPeer, AxonaDomain, NeuronNode, AxonaManager,
  deriveIdentity, deriveTopicId,
  WIRE_VERSION, KERNEL_VERSION,
} from '@axona/protocol';
import { WebSocket } from 'ws';   // Node only; browsers have global WebSocket

const BRIDGE_URL = 'wss://bridge.axona.net';

// 1. Derive a 264-bit Ed25519 identity in the us-east S2 cell.
//    The bottom 256 bits are deterministic from the keypair we generate
//    here - see §4 for stable-identity strategies.
const identity = await deriveIdentity({ lat: 38.0, lng: -77.0 });
console.log('My nodeId:', identity.id);   // 66-char hex
console.log('My region S2 prefix:', identity.id.slice(0, 2));   // "df"

// 2. Build the local DHT node + AxonaManager (the pubsub engine).
const node    = new NeuronNode({ id: BigInt('0x' + identity.id),
                                 lat: 38.0, lng: -77.0 });
const domain  = new AxonaDomain({ k: 20 });
const peer    = new AxonaPeer({ domain, node, identity });
await peer.start();

// 3. Connect to the bridge over WebSocket and tunnel Axona frames
//    through it. (Production peers also open WebRTC mesh channels;
//    the bridge alone is enough for a working demo.)
//    The bridge's WebSocket protocol is documented in
//    implementation/Axona-Wire-Protocol-v0.71.md.

// (full transport setup omitted in this snippet - see the runnable
//  example at examples/minimal-pubsub/ in this repo for the complete
//  client.js, which is ~120 lines.)
```

The runnable, complete version of this is in `examples/minimal-pubsub/` — open it, run `node publisher.js` in one terminal and `node subscriber.js` in another, and you'll see a roundtrip in under a minute.

2.3 Or: just use the deployed peer

If you want to see Axona working end-to-end without writing any code, open https://axona.net in two browser tabs. Each tab is an independent peer. Add a subscription in one tab, publish from the other, and the message arrives.

The full source of that reference peer is in `axona-peer/` and is the canonical example of a complete browser application — about 1500 lines of JavaScript including UI.

---

3. Mental model

3.1 Peers and the mesh

Every participant is a `Peer` — a long-lived process with:

- **An identity**: a 264-bit `nodeId` and an Ed25519 keypair.
- **A synaptome**: an adaptive routing table of other peers it knows about, weighted by recency, latency, and usefulness.
- **A transport**: a way to talk to other peers. In the browser this is a `CompositeTransport` that combines a WebRTC mesh (peer-to-peer data channels) with a WebSocket fallback through the bridge.

Peers form a **mesh** — a graph of bidirectional channels. The mesh is NOT a global broadcast topology. Each peer only directly knows ~5–50 others; everything else is reached by routed forwarding.

3.2 Topics, axons, and replication

A **topic** is just a hashed name. When you call `peer.pub(name, ...)`, the protocol:

1. Derives a 264-bit `topicId` from your name (and optionally a publisher ID — see §5).
2. Finds the **K closest peers** to that `topicId` in XOR space (`K` defaults to 5).
3. Sends a `pubsub:publish-k` frame directly to each of them. Each one appends the message to its local replay cache for that topic and forwards it to any subscribers it's tracking as children.

Those K peers are called the topic's **axons**. They aren't a privileged server set — any peer that happens to be in the K-closest set for that topic ID acts as an axon for it, and the set rotates as nodes join/leave.

A peer that calls `peer.sub(name, handler)` sends `pubsub:subscribe-k` to the K closest peers and registers itself as a child of each. From that point on, any publish to the topic fans out to it. If the subscriber asks `{since: 'all'}` at subscribe time, the axon also sends its replay cache.

3.3 Geographic clustering

The S2 prefix on the top 8 bits of every ID means that:

- Two peers in the same S2 cell have the same top 8 bits → XOR distance between them is 0 in that range.
- A topic ID whose top 8 bits match a region → peers in that region are XOR-close to it → they're naturally selected as axons.

You usually want this. A "us-east/world-news" topic whose top 8 bits are us-east (`df`) will land its axon set on us-east peers, where the cluster of subscribers is also likely to live, minimizing cross-region hops. This is the **region-keyed topic** pattern documented in §5.3.

3.4 Signed envelopes

Every publish is wrapped in an envelope:

```
{
  msgId:        '0xabcd…',         // sha256 of (topic|ts|message|publisher)
```

```
  ts:           1779306455550,
  topic:        'us-east/world-news',
  message:      'whatever-you-pub',
  signerPubkey: '<64-char hex>',   // present iff signed
  signature:    'ed25519:<128 hex chars>',
}
```

Signing is on by default; subscribers receive the envelope as-is and can verify it locally with `verifyEnvelope(envelope)`. The protocol does not mandate a specific identity scheme — applications choose whether to gate delivery on signature verification.

You can also pass `{sign: false}` to `peer.pub` for unsigned messages — useful for genuinely public, anonymous topics.

3.5 What the protocol secures (and what it doesn't)

Axona is **self-authenticating**: every guarantee is enforced by cryptography the peers carry themselves — there is no certificate authority, central trust server, or reputation service. A node's identity *is* its keypair. As of kernel v2.16.0:

- **Identity is provable, not asserted.** A nodeId's bottom 256 bits are `SHA-256(pubkey)`, and every connection runs the `axona/5` handshake: the peer proves it holds the key (Ed25519 signature), and the proof is bound to *that specific live connection* so it can't be replayed onto another link. A peer cannot claim an id it doesn't own.
- **The bridge is trusted for introductions only.** It bootstraps peers and relays WebRTC signaling, but mesh links bind their handshake to the connection's DTLS fingerprint, so the bridge cannot transparently sit in the middle of "direct" peer traffic.
- **You can only act for yourself.** A peer can subscribe only *itself* (no naming a victim as the subscriber), nodes only host topics they're routing-responsible for, and publisher signatures are checked at the topic's ingress before fan-out.
- **The routing table is earned.** A peer becomes one of your routing hops only after you've connected to it and it has authenticated — so gossip can suggest peers but cannot inject attacker-controlled hops (eclipse resistance). You can watch this via `peer.health().meshDegraded` and `boundCount` (§8.3).
- **Messages are fresh, ordered, and yours to control.** Every signed publish carries a per-publisher `seq` + timestamp under its signature, so a captured message can't be replayed as live once it ages out, and topics have a deterministic order (v2.9.0). You can **retract** a message you signed and **remove** a topic you own — each authorized by that same keypair, no gatekeeper (v2.10.0, §6.6). On open topics a per-publisher quota stops one party flooding out the rest.

What it does NOT do: encrypt your application payload — `message` is opaque bytes; if you need confidentiality, encrypt inside it end-to-end. And authentication proves *who* a peer is, not that what it *says* about the network is true beyond what the cryptography checks. The full versioned record is the security changelog.

For most apps this is transparent — `peer.pub` signs, the network verifies, and you just check `(await verifyEnvelope(env)).ok` on what you consume (§6.2). The guarantees are there whether or not you think about them.

---

4. Identity

4.1 Deriving an identity

```
import { deriveIdentity } from '@axona/protocol';

const identity = await deriveIdentity({ lat: 38.0, lng: -77.0 });
// {
```

```
//   id:         'df27c92bb6184290…' (66-char hex),
//   pubkey:     Uint8Array(32),
//   pubkeyHex:  '<64-char hex>',
//   privateKey: <Web Crypto CryptoKey, Ed25519>,
//   region:     { lat: 38.0, lng: -77.0 },
//   createdAt:  1779306455550,
// }
```

The top 8 bits of `identity.id` are `geoCellId(lat, lng, 8)` — the S2 cell at level 8 (~256 buckets covering Earth). The bottom 256 bits are `sha256(pubkeyBytes)`. Both are deterministic given the inputs; the random element is the Ed25519 keypair.

`deriveIdentity` always mints a fresh key (pass `extractable: true` if you need to export it). To **reuse** an identity across runs, serialize it and restore — don't try to inject a keypair into `deriveIdentity`:

```
const identity = await deriveIdentity({ lat: 38.0, lng: -77.0, extractable: true });
const blob     = await dumpIdentity(identity);   // JSON-safe envelope
const same     = await loadIdentity(blob);        // restores the same nodeId
```

4.2 Persisting an identity

Web Crypto `CryptoKey` is opaque — you can't `JSON.stringify` it. The kernel ships `dumpIdentity` / `loadIdentity` which round-trip through a **PKCS#8 envelope**:

```
import { dumpIdentity, loadIdentity } from '@axona/protocol';

// Save
const envelope = await dumpIdentity(identity);
// envelope = {
//   id:         '<66 hex>',
//   pubkey:     '<64 hex>',
//   privkey:    '<base64 PKCS#8>',
//   region:     { lat, lng },
//   createdAt: ...,
// }
localStorage.setItem('my-identity', JSON.stringify(envelope));

// Load
const stored   = JSON.parse(localStorage.getItem('my-identity'));
const identity = await loadIdentity(stored);
```

In the browser, `dumpIdentity` requires Web Crypto to be available (Chrome 110+, Safari 17+, Firefox 130+). In Node 20+ it's also built in.

The pitfall to know about: `axona-peer` itself currently does NOT persist the kernel envelope — it stores only `{lat, lng, region}` in sessionStorage and re-derives a fresh keypair on every page load. That's a per-session deliberate choice for the testing-era UX. If your app needs stable IDs across sessions, persist the PKCS#8 envelope yourself as shown above.

4.3 Region selection

The S2 prefix determines the routing neighborhood. Three strategies:

Strategy	When
Hardcode `{lat, lng}`	Server-side peer (bridge, daemon, scraper) where region is known.

Strategy	When
Geolocate the user	Browser app with `navigator.geolocation.getCurrentPosition`. The reference peer falls back to a region picker if geolocation is denied.
Let the user pick	A region dropdown; `axona-peer` ships a 15-region table you can copy from `axona-peer/src/identity.js`.

Whatever the choice, the S2 prefix is just a routing hint — you can publish to any region's topics regardless of where your peer lives (see §5.3).

---

5. Topics

5.1 Three addressing modes

A topic ID is always `<8-bit S2 prefix><256-bit hash>` (66 hex chars). Where each part comes from depends on the **addressing mode**:

Mode	`deriveTopicId(publisherArg, name)`	Top 8 bits	Hash
Public	`deriveTopicId(null, name)`	`00` (global bucket)	`sha256(name)`
Publisher-keyed	`deriveTopicId(myFullNodeId, name)`	publisher's S2 prefix	`sha256(myNodeId + ':' + name)`
Region-keyed	`deriveTopicId(synthRegionId, name)`	region's S2 prefix	`sha256(synthRegionId + ':' + name)`

You pick the mode by what you pass to `opts.publisher` in `peer.pub` / `peer.sub`:

```
peer.pub(name, msg)                                  // publisher-keyed, my own ID
peer.pub(name, msg, { publisher: null })             // public, '00' bucket
peer.pub(name, msg, { publisher: otherPeerFullId })  // publisher-keyed under another peer
peer.pub(name, msg, { publisher: synthRegionId })    // region-keyed
```

5.2 Public mode

```
const topicId = await deriveTopicId(null, 'world-news');
// topicId = '00' + sha256('world-news')
```

Top 8 bits are always `00`. The global bucket is essentially a single giant address space with no regional clustering — every peer is equally "far" from every public topic. Use it for:

- Genuinely planetary topics (a global announcement channel).
- Tests where you don't care about locality.

Don't use it for region-scoped channels — you'll lose the locality benefit and your axon set will be effectively random.

5.3 Region-keyed topics (the most common case)

For most chat / feed / forum applications you want **region-keyed** topics. The trick is to use a **synthetic publisher ID** whose top 8 bits encode the region:

```
import { geoCellId } from '@axona/protocol';

function regionSynthPublisher(lat, lng) {
  const s2 = geoCellId(lat, lng, 8);
  // 2 hex chars (S2 cell) + 64 zero hex chars = 66 chars total
  return s2.toString(16).padStart(2, '0') + '0'.repeat(64);
}

const usEast    = regionSynthPublisher(38.0, -77.0);  // 'df00…00'
const topic     = 'world-news';

const msgId = await peer.pub(topic, 'hi', { publisher: usEast });
// topicId derives to 'df' + sha256(usEast + ':' + 'world-news')
```

Every peer that picks the same `(lat, lng)` derives the same synth ID deterministically, so publishers and subscribers compute identical topic IDs. The synth ID never signs anything — it's purely an addressing device; the envelope is signed by your real identity.

5.4 Publisher-keyed topics

If a topic belongs to one specific publisher (a user's personal feed, a brand's broadcast channel), pass the publisher's real nodeId hex:

```
const aliceFeed = await peer.sub('posts',
  envelope => { /* render Alice's post */ },
  { publisher: aliceFullNodeIdHex });
```

The top 8 bits of `topicId` are then Alice's S2 prefix, and Alice's peers naturally form the axon set for her feed.

5.5 Designing topic names

`topicName` is an opaque string from the protocol's perspective — it's just hashed. Conventions used by `axona-peer`:

- For region-keyed topics, name = `'<regionId>/<eventName>'`. The `regionId` (e.g. `"us-east"`) is there for the UI / persistence layer; the protocol doesn't care.
- For publisher-keyed topics, name = whatever your app conventions prefer; e.g. `"posts"`, `"likes"`, `"profile-update"`.

Keep names short and stable. Renaming a topic creates a new topic ID with empty replay cache.

---

6. Publish and subscribe

6.1 The unified `peer.pub` / `peer.sub` API

```
// Publish - returns the envelope's msgId
const msgId = await peer.pub(topicName, message, {
  publisher: <null | hex>,   // addressing mode (§5)
  sign:      true,           // default; pass false for anonymous
});

// Subscribe - returns a Subscription handle with a .stop() method
const sub = await peer.sub(topicName, (envelope) => {
  // envelope = { msgId, ts, topic, message, signerPubkey?, signature? }
  console.log(envelope.message);
```

```
}, {
  publisher: <null | hex>,   // must match the publisher's mode
  since:     'all',          // see §6.3
});

// ...

await sub.stop();
```

Crucial: `peer.pub` and `peer.sub` MUST use the same `opts.publisher` for the topic ID to match. Mismatch = same name, no delivery (silent — the IDs are just different).

6.2 The envelope

```
{
  msgId:        '<64-char hex sha256>',
  ts:           <ms since epoch>,
  topic:        '<the topic name string you published>',
  message:      <any JSON-serializable value>,
  signerPubkey: '<64-char hex>',     // present iff signed
  signature:    'ed25519:<128 hex>', // present iff signed
}
```

`message` can be any JSON-serializable value — string, number, object, array, nested whatever. The kernel JSON-encodes the entire envelope at the wire layer.

Verify signatures yourself if your app cares:

```
import { verifyEnvelope } from '@axona/protocol';

const sub = await peer.sub(topic, async (env) => {
  if (env.signature) {
    const res = await verifyEnvelope(env);   // returns { ok, reason, signed }
    if (!res.ok) {
      console.warn('rejected forged envelope', env.msgId, res.reason);
      return;
    }
  }
  appendToFeed(env);
});
```

> `verifyEnvelope` returns a **result object**, not a boolean — test `.ok`. Writing `if (!(await verifyEnvelope(env)))` is always false (an object is truthy), so that guard would silently never reject anything.

`verifyEnvelope` reconstructs the canonical signing input and verifies against `env.signerPubkey`. It does NOT decide whether you trust that pubkey — your application controls that (whitelist, web of trust, TOFU, etc.). As of kernel v2.7.0 the network *also* verifies publisher signatures at the topic's ingress before fan-out, so spoofed-signature traffic is dropped mesh-side — but verifying what you consume is still good practice.

6.3 Replay vs live tail (`since`)

`peer.sub`'s `since` option controls what the axon delivers right after subscribe registration:

`since`	Behavior
omitted / `undefined`	Live tail only — no cached messages.

since	Behavior
'all'	Replay every message in the axon's cache, then live tail.
'latest'	Replay only the most recent ~1 second of cache, then live tail.
<number>	Replay messages with publishTs > since (ms epoch).

For chat / forum UX you almost always want since: 'all' — new subscribers expect to see history.

6.4 Queue size and hold time

Each topic's replay queue is bounded (kernel v2.10.0):

- **Max messages** — default **100**, ceiling **256**. When the queue is full the **lowest-ordered** message is evicted, ordered by the signed seq (then ts, msgId). Because the order comes from signed fields, every replica evicts the *same* message — caches stay consistent. A max of **1** is a *retained / latest-value* slot: each new publish replaces the prior one (pair it with peer.pull(null, …) for "give me the current value").
- **Hold time** — default **24 h**, hard ceiling **48 h**. A message past its hold expires, is swept, and stops being delivered or pulled. A peer.pull slides the hold forward (now + hold), bounded by the ceiling — reads keep a hot message alive but can't pin it forever.
- **Per-publisher quota (open topics only)** — on a public/anyone-may-publish topic one publisher can hold at most ¼ of the queue, so a single flooder can't evict everyone else. Owner-gated topics aren't quota'd.

The cache is in-memory and disappears when an axon disconnects or restarts. A topic with no children for rootGraceMs (default 60 s) is swept by the maintenance loop — so if no one is subscribed for >60 s, replay history can be lost before its hold time even elapses.

6.5 Subscription renewal (soft-state leases)

A subscription is **soft state**, not a permanent registration. You call peer.sub() **once**; the kernel keeps it alive for you by periodically re-announcing it to the topic's current root axons. You never call sub() again — renewal is automatic and idempotent: a renewal just bumps a timestamp on the root, it does not re-deliver cached messages.

Two timers govern the lease (both are AxonaManager options, with the defaults below):

Constant	Default	Role
refreshIntervalMs	10 s	How often the kernel re-announces every local subscription to the topic's K-closest roots.
maxSubscriptionAgeMs	30 s	How long a root keeps a subscriber it hasn't heard a renewal from. Past this, the root drops it.

So in steady state a subscriber renews **every 10 seconds**, and a root forgets a subscriber it hasn't heard from in **30 seconds**. The 3x headroom is deliberate: a subscriber can miss up to two consecutive renewals — a transient mesh hiccup, a dropped frame, a brief disconnect — without being evicted, and the next renewal re-establishes the lease.

This cadence is also what lets the system self-heal around churn. Because every subscription re-anchors on the ~10 s tick onto the *current* root set (the refresh flushes the K-closest cache first), a root — or the bridge — leaving is absorbed within a cycle or two: subscribers simply re-home onto the surviving roots on their next renewal. A peer that leaves *gracefully* announces its departure, so subscribers re-anchor immediately rather than waiting for the tick.

For your application this means: nothing to manage — you don't renew subscriptions yourself. Two implications are worth knowing:

- A subscription does **not** survive on its own if your peer goes silent longer than `maxSubscriptionAgeMs`. When the peer comes back the next refresh re-subscribes automatically, and `since`-replay backfills anything missed that's still within the topic's hold time (§6.4).
- Renewal is steady background traffic — roughly one re-announce per subscription per `refreshIntervalMs`, to K roots. It's negligible at normal scale, but a topic with very many subscribers is a reason to consider a longer interval (both timers are tunable when you construct the peer's `AxonaManager`).

6.6 Publishing flow (what happens under the hood)

1. `peer.pub` builds a signed envelope, JSON-encodes it.
2. `am.pubsubPublish(topicId, json, meta)` is called on the local `AxonaManager`.
3. `_asyncPublish` calls `findKClosest(topicId, K)` to identify the axon set.
4. For each axon, `dht.sendDirect(peerId, 'pubsub:publish-k', {...})` fires a direct frame. If the peer is bound (WebRTC or bridge WS), it goes direct; otherwise the transport routes it.
5. Each axon's `_onPublishDirect` handler caches the message (creating a role on the fly if needed) and forwards to any children registered for the topic.

The publisher's own AxonaManager does NOT cache the publish unless its own ID is among the K-closest. If you publish and want to be sure your own subsequent `peer.sub` sees the message, either subscribe first or include yourself in the dht adapter's findKClosest result (see §13).

6.7 Message lifecycle: unsubscribe, retract, keep-alive, remove

Beyond pub/sub, the kernel gives you four lifecycle controls, all **self-authenticating** — the authority to act is proven by the same keypair that proved authorship/ownership, with no central gatekeeper:

```
// Leave a topic (counterpart to sub; stops all your local subs for it).
await peer.unsub('rooms/london', { publisher: synth });

// Retract a message YOU published ("unsend"). Creator-only: the network
// accepts it only if signed by the same key that signed the original.
await peer.kill('rooms/london', msgId, { publisher: synth });

// Keep a message alive past its default hold (touch  v2.15.0; ownership
// gate  v2.16.0). Open topic → ANYONE may; owned topic → owner-only. Resets
// the hold (bounded by the 48 h ceiling) and moves the message to the head of
// the queue so it's evicted last. No re-publish needed.
await peer.touch('rooms/london', msgId, { publisher: synth });

// Remove a topic's whole queue. OWNER-only (the identity whose nodeId
// seeds the topic id). `{ destroy: true }` removes the topic entirely.
await peer.unpub('rooms/london', { destroy: true });
```

`touch` is the keep-alive counterpart to `pull`'s read-slide (§6.4): an explicit refresh that doesn't require reading the message. Like a pull it can extend a message's life but never past the absolute 48 h ceiling from first publish — so it can't pin a message forever.

Subscribers see a retraction as a **delete marker** on their existing handler, so branch on it:

```
await peer.sub('rooms/london', (env) => {
  if (env.deleted) { ui.remove(env.msgId); return; }   // a kill arrived
  ui.render(env.message);
}, { publisher: synth, since: 'all' });
```

Two honesty notes worth designing around:

- `kill` is **best-effort redaction, not a cryptographic un-send** — a subscriber that already has the plaintext can keep it, and an offline one may never see the purge. An **anonymous (`sign: false`) message can't be killed** at all (no provable creator).
- `unpub` is **owner-only** and unavailable on public (ownerless) topics.

---

7. Direct messaging

Not all peer-to-peer communication is pub/sub. For 1:1 messages, `AxonaPeer` exposes a direct-message API. The kernel uses one handler per peer (called for every inbound direct message); your application dispatches on payload shape if you want multiple message types.

```
// Request/response - Alice asks Bob, gets a reply.
const reply = await alice.send(bobNodeIdHex, { kind: 'ping', body: 'hello' });
// reply is whatever Bob's handler returned.

// Fire-and-forget - Alice notifies Bob, no reply expected.
await alice.notify(bobNodeIdHex, { kind: 'typing', user: 'alice' });

// Bob's single handler - invoked for every inbound direct message.
// senderId is the sender's 66-char hex nodeId (set by the transport
// at the receiving end, after channel handshake).
bob.onMessage(async (senderId, message) => {
  switch (message?.kind) {
    case 'ping':
      return { reply: 'pong' };           // becomes Alice's send() resolution
    case 'typing':
      console.log(`${senderId} is typing`);
      return;                              // notify() discards return values
    default:
      return { error: 'unknown kind' };
  }
});
```

- **Target ID is 66-char hex** (the same string `identity.id` returns).
- Messages are routed through the synaptome — if Alice and Bob aren't directly meshed, the routing layer walks intermediate hops.
- `senderId` is set by the transport at the receiving end. You can trust it for routing purposes; verify application-layer identity yourself if security matters (`message` can carry a signed envelope).
- `peer.onMessage` is **single-handler**: calling it twice replaces the previous handler. To support many message kinds, dispatch on `message.kind` (or `.type`, or whatever convention your app picks) inside the one handler.
- `send`/`notify` do **not** sign by default the way `pub` does. If you need end-to-end authenticity for DMs, wrap your payload with `buildEnvelope({ ..., sign: true })` before sending and verify on receipt with `verifyEnvelope`.

---

8. Mesh introspection and discovery

8.1 Who's online

```
peer.peers();    // returns ['<66-char hex>', '<66-char hex>', ...]

const unsubJoin = peer.onPeerJoin((peerId, ctx) => {
  console.log('hi', peerId);              // peerId is 66-char hex
});

const unsubLeave = peer.onPeerLeave((peerId, ctx) => {
  console.log('bye', peerId);
});

// Both callbacks return an unsubscribe function:
unsubJoin();
```

`peers()` returns the hex nodeIds currently in this peer's synaptome (roughly: peers we have a live channel to, plus a few we've learned about via lookups but aren't directly meshed with). In a small mesh this is everyone; in a large mesh it's a constant-sized subset.

8.2 Looking up a specific peer

```
// `targetKey` is whatever ID space your transport routes in.  With
// axona-peer's wiring (NeuronNode.id stored as BigInt), pass a BigInt:
const result = await peer.lookup(BigInt('0x' + targetHex));
// {
//   path:  [<bigint>, <bigint>, ...],   // the hop sequence
//   hops:  3,
//   time:  47,                           // ms (live RTT, see v1.1.2 notes)
//   found: true,
// }
```

`lookup` is an iterative XOR-routed walk. It's how the kernel finds peers it doesn't directly know about. As a side effect, it warms the local synaptome with the discovered route.

8.3 Health

```
peer.health();
// {
//   nodeId, synaptomeSize, peers: [...], subscriptions,
//   axonRoles: [{ topic, isRoot, children, cacheSize }],
//   wireVersion, started,
//   transport: {           // null on sim/node; populated on the web transport
//     boundCount,          // peers authenticated via axona/5
//     meshChannels, meshOpen, meshBound,   // raw vs open vs authenticated channels
//     bridgeState,
//   },
//   meshDegraded,          // true   channels open but NOT yet authenticated
// }
```

`meshDegraded` is the routing-truth signal — distinguish "channels are open" from "channels are open *and* carrying authenticated routing." A sustained `true` (across several polls) means the mesh looks connected but isn't actually routing; `boundCount`/`meshBound` are the honest usable-peer counts. Cheap; safe for a status dashboard. See the API reference §8 health() for the full shape.

8.4 Logs and errors

```
// onLog takes the LEVEL first, then the handler:
peer.onLog('warn', (msg, ctx) => { /* … */ });   // 'debug'|'info'|'warn'|'error'
peer.onError((err) => { /* … */ });                // background AxonaError
peer.onUpgradeRequired((err) => { /* show upgrade banner */ });
```

The protocol emits structured log events for routing decisions, admissions, axon recruitments, etc. Most applications don't need them; useful when debugging mesh behavior.

---

9. Pull and metrics

9.1 Pull — fetch a single envelope, or the latest

```
const envelope = await peer.pull(msgId, {
  topic:     'us-east/world-news',
  publisher: regionSynth,
  timeoutMs: 1000,
});
// envelope or null if not in any reachable axon's cache window

// Or, with no msgId (kernel  v2.10.0), the topic's most-recent message:
const latest = await peer.pull(null, { topic: 'us-east/world-news', publisher: regionSynth });
```

`pull` is useful when you have a `msgId` but missed the live delivery (e.g., a reshare link). It queries the K-closest axons for that topic and returns the cached envelope. Pass **`null` instead of a `msgId`** to fetch the latest message (highest signed `seq`) — handy with a `maxMessages: 1` retained-slot topic for "current value" reads. A successful pull also slides the message's hold time forward (bounded by its 48 h ceiling). A pull never returns an **expired** message (past its hold) — that's a `null`, same as a cache miss.

9.2 Metrics — coarse delivery counters

```
const m = await peer.metrics('us-east/world-news', {
  publisher: regionSynth,
  timeoutMs: 500,
});
// {
//   publishes:     <count of distinct messages ever seen>,
//   current_count: <events live in the tree right now>,   // kernel  v2.11.0
//   subscribers:   <max direct subscribers at a relay>,   // live, kernel  v2.11.0
//   deliveries, pulls, reshares, relayCount,
// }
```

Metrics are best-effort — they're aggregated from whichever axons respond within the timeout. `current_count` is the live retained-event count (drops as messages are killed or expire); `publishes` is cumulative (only rises). Don't use them for billing; do use them for "X people are subscribed to this room" UX.

As of kernel v2.14.0 the request fans out to the topic's **whole K-closest root set** — the same set publishes replicate to — and merges the replies, so the counts reflect the full tree rather than a single root. (Before v2.14.0 the query hit only the closest root, which in a churning mesh could report `current_count: 0` even while new subscribers were getting a full replay from a sibling root — purely a measurement artifact, since delivery itself was fine.)

Access follows ownership (kernel v2.12.0): an **unowned** topic — public (`publisher: null`) or synthetic region-keyed (`prefix‖0…0`, ownerless by construction) — exposes its metrics to anyone, which is why the

region-keyed rooms in this guide are freely inspectable. An **owner-keyed** topic answers only its owner; everyone else gets zeroed counters, not the owner's data.

---

10. Persistence

10.1 The `PersistenceAdapter` contract

The kernel does not assume any storage layer. You pass it a `PersistenceAdapter` and it round-trips state through it:

```
import { PersistenceAdapter } from '@axona/protocol';

class MyAdapter extends PersistenceAdapter {
  async load(key)        { /* return previously-saved bytes or null */ }
  async save(key, bytes) { /* persist */ }
  async clear()          { /* wipe */ }
}
```

The kernel ships two ready-made adapters, as **classes** behind sub-path imports (so neither pulls `indexedDB` into Node nor `node:fs` into the browser bundle):

- **`IndexedDBPersistence({ dbName })`** — browser, IndexedDB — `@axona/protocol/persistence/indexeddb.js`
- **`FilePersistence({ dir })`** — Node, JSON files under `dir` (default `./.axona`) — `@axona/protocol/persistence/fi`

Pass an instance as the constructor's **`persist`** option:

```
import { IndexedDBPersistence } from '@axona/protocol/persistence/indexeddb.js';

const peer = new AxonaPeer({
  domain, node, identity,
  persist: new IndexedDBPersistence({ dbName: 'my-app' }),
});
await peer.start();
```

State persisted automatically: identity envelope (if dumped), active subscriptions (so they restore on reload), synaptome snapshot. The checkpoint cadence is internal — apps don't need to drive it.

10.2 Manual snapshots

For testing / migration:

```
const state = await peer.snapshot();
// state = { identity?, subscriptions, synaptome, ... }

// fromSnapshot is a STATIC factory, not an instance method:
const restored = await AxonaPeer.fromSnapshot(state, { engine, node, transport });
```

This is the same data the auto-persister uses; you can store it anywhere you like.

---

11. The bridge

The bridge is a Node service. It does three things:

1. **Signaling for WebRTC**: peers exchange SDP offers/answers/ICE through it on a per-pair basis.
2. **Peer-list distribution**: when a peer connects, the bridge sends the current connected peer list so the new peer can initiate WebRTC offers.

3. **Universal-hub peer**: the bridge runs its own `AxonaPeer` and is meshed to every connected browser. It acts as a routing hop for peers that can't directly reach each other.

11.1 Using the public bridge

For most demos / small apps, use `wss://bridge.axona.net` — it's production, has TURN, and accepts MIN_PEER_VERSION 1.1.0. Your peer just opens a WebSocket and follows the handshake protocol.

11.2 Running your own bridge

If you need privacy / control / a different MIN_PEER_VERSION, clone axona-bridge:

```
git clone https://github.com/axona-net/axona-bridge
cd axona-bridge
npm ci
PORT=8080 npm start
```

Locally, peers can connect at `ws://localhost:8080`. For production, put nginx + certbot in front to terminate TLS. Full deploy recipe is in `axona-bridge/deploy/README.md`.

11.3 Bridge environment variables

Var	Purpose	Default
`PORT`	TCP port for the WS server	8080
`MIN_PEER_VERSION`	Reject peers below this	`1.1.0`
`BRIDGE_LAT`, `BRIDGE_LNG`	The bridge's own region	`38.0, -77.0`
`BRIDGE_REGION_LABEL`	Cosmetic	derived
`BRIDGE_IDENTITY_PATH`	Where to persist the bridge's Ed25519 envelope	`bridge-identity.json`
`TURN_AUTH_SECRET`	Shared secret for time-bound TURN credentials (optional)	(none)
`LOG_LEVEL`	`error` / `warn` / `info` / `debug`	`info`

11.4 Bridge wire protocol

The bridge accepts JSON frames over a WebSocket. Three frame types matter for most apps:

```
// from server → peer on connect
{ type: 'welcome', id: <connId>, version: '1.1.0',
  turn: { urls: [...], username, credential } }

// from server → peer
{ type: 'peer-list', peers: [{ id, nodeId, region }, ...] }

// from peer → server (request)
// from server → peer (relay)
{ type: 'axona', payload: { k: 'req'|'ntf'|'res', type, body, ... } }
```

The `'axona'` envelope tunnels protocol-layer frames (hello, hello-ack, publish-k, subscribe-k, …). Full spec: Axona Wire Protocol v0.71.

---

12. Worked example: a regional chat app

Let's build a chat application called **AxonaTalk**: users pick a region, join chat rooms within their region, see message history, send DMs.

12.1 Topic plan

```
// Chat room: region-keyed so us-east users cluster on us-east axons.
//   topicName = `${regionId}/room/${roomName}`
//   publisher = synth ID derived from the region's lat/lng
//
// Direct message: peer.send, no pub/sub.
//
// User presence: each peer announces itself by publishing a "hello"
// envelope to a per-region presence topic on join.
//   topicName = `${regionId}/presence`
```

12.2 The message shape

```
// Chat message envelope.message:
{ kind: 'chat',     author: '<nodeIdHex>', text: 'hi',  replyTo: null }

// Presence ping:
{ kind: 'presence', nodeId: '<hex>',     name: 'alice', lat, lng }

// DM (sent via peer.send, signed manually):
{ kind: 'dm',       author: '<hex>',     text: '…', envelope: <signedEnv> }
```

12.3 Boot

```
// boot.js
import {
  AxonaPeer, AxonaDomain, NeuronNode,
  deriveIdentity, dumpIdentity, loadIdentity,
  geoCellId,
} from '@axona/protocol';

const REGIONS = [
  { id: 'us-east',   label: 'US East',        lat: 38.0,  lng: -77.0 },
  { id: 'us-west',   label: 'US West',        lat: 37.0,  lng: -122.0 },
  { id: 'eu-west',   label: 'EU West',        lat: 48.9,  lng: 2.3 },
  // ...
];

function regionSynthPublisher(lat, lng) {
  const s2 = geoCellId(lat, lng, 8);
  return s2.toString(16).padStart(2, '0') + '0'.repeat(64);
}

async function boot() {
  // Load or derive identity.
  const stored = localStorage.getItem('axonatalk.identity');
  const region = REGIONS.find(r => r.id === localStorage.getItem('axonatalk.region')) ?? REGIONS[0];

  let identity;
  if (stored) {
    try { identity = await loadIdentity(JSON.parse(stored)); }
    catch { /* fall through, re-derive */ }
  }
  if (!identity) {
```

```
    identity = await deriveIdentity({ lat: region.lat, lng: region.lng });
    localStorage.setItem('axonatalk.identity',
                         JSON.stringify(await dumpIdentity(identity)));
  }

  const node    = new NeuronNode({ id: BigInt('0x' + identity.id),
                                   lat: region.lat, lng: region.lng });
  const domain  = new AxonaDomain({ k: 20 });
  const peer    = new AxonaPeer({ domain, node, identity });

  // Wire up your transport (CompositeTransport for browser; see
  // axona-peer/src/axona_node.js for the full setup).
  // Once transport is started:
  await peer.start();

  return { peer, identity, region };
}
```

12.4 Join a room

```
function chatTopic(region, roomName) {
  return {
    name:      `${region.id}/room/${roomName}`,
    publisher: regionSynthPublisher(region.lat, region.lng),
  };
}

async function joinRoom(peer, region, roomName, onMessage) {
  const { name, publisher } = chatTopic(region, roomName);
  return await peer.sub(name, env => {
    if (env.message?.kind !== 'chat') return;
    onMessage(env.message, env.ts, env.msgId);
  }, { publisher, since: 'all' });
  // Returns a Subscription - call .stop() to leave.
}
```

12.5 Send a chat message

```
async function sendChat(peer, region, roomName, text) {
  const { name, publisher } = chatTopic(region, roomName);
  const msgId = await peer.pub(name, {
    kind:    'chat',
    author:  peer.getNodeId(),
    text,
    replyTo: null,
  }, { publisher });
  return msgId;
}
```

12.6 Presence (who's in this region)

```
async function announcePresence(peer, region, displayName) {
  const topic     = `${region.id}/presence`;
  const publisher = regionSynthPublisher(region.lat, region.lng);
```

```
  // Subscribe first so we hear other announcements.
  const sub = await peer.sub(topic, env => {
    if (env.message?.kind !== 'presence') return;
    addUserToOnlineList(env.message);
  }, { publisher, since: 'latest' });

  // Republish our own presence on a timer so late joiners learn we exist.
  setInterval(() => {
    peer.pub(topic, {
      kind:   'presence',
      nodeId: peer.getNodeId(),
      name:   displayName,
      lat:    region.lat, lng: region.lng,
    }, { publisher });
  }, 15_000);

  return sub;
}
```

12.7 Direct messages

```
async function sendDM(peer, recipientHex, text) {
  return await peer.send(recipientHex, {
    kind:   'axonatalk:dm',
    author: peer.getNodeId(),
    text,
    ts:     Date.now(),
  });
  // Returns whatever the recipient's onMessage handler resolved to -
  // e.g. an ack receipt with the recipient's view of `ts`.
}

function registerDMHandler(peer, onDM) {
  // peer.onMessage is single-handler - if your app uses multiple
  // direct-message kinds, dispatch on payload.kind here.
  peer.onMessage(async (senderId, message) => {
    if (message?.kind === 'axonatalk:dm') {
      onDM({ from: senderId, text: message.text, ts: message.ts });
      return { ack: true, receivedAt: Date.now() };
    }
    // ...handle other kinds...
  });
}
```

12.8 What you don't have to build

You don't have to:

- Build a server. The bridge is shared infra; your peers run in browsers and talk to each other.
- Build presence diffing — re-subscribing to the presence topic with `since: 'latest'` gives you everyone who pinged in the last few seconds.
- Build replay/history — `since: 'all'` gives you the cache window.
- Build a message bus — the synaptome routes for you.
- Build cryptography — `peer.pub` signs envelopes; `verifyEnvelope` checks them.

You do have to:

- Decide what `displayName` means in your social model (real names? handles?).
- Decide trust: who can post in which rooms? The protocol delivers signed envelopes; you reject ones whose signerPubkey isn't in your app's allowlist (or accept everyone, your call).
- Render the UI. None of this is your problem at the protocol level.

---

13. Common pitfalls

These are real bugs we hit while building the reference peer. Knowing them ahead saves hours.

13.1 Topic mismatch from publisher arg

`peer.pub` and `peer.sub` MUST use the same `opts.publisher` value. If one passes `null` (public mode) and the other passes a synth region ID, they compute different topic IDs and never see each other's messages. There's no warning — the IDs are simply different.

Symptom: live tail works for self-pub-self-sub in the same tab, but cross-peer delivery fails silently.

13.2 Identity isn't persisted by default

`axona-peer` re-derives the keypair on every page load (region is preserved). If your app depends on stable nodeIds across sessions — for "block this user" lists, persistent DM threads, etc. — persist the kernel envelope yourself with `dumpIdentity` / `loadIdentity`.

13.3 Wait for mesh handshakes before pub/sub

`AxonaPeer.start()` returns as soon as the local state is set up. Mesh handshakes (the `axona:hello` → `axona:hello-ack` round-trip that admits peers to the synaptome) complete asynchronously over the next few seconds. If you publish immediately, your `findKClosest` returns only yourself + the bridge, and your axon set is incomplete.

Recommended pattern: wait for `peer.peers().length >= expectedMin` before allowing the user to publish, or just delay the first publish ~2–3 seconds after `start()`.

13.4 The K-closest reachability problem

The kernel's `findKClosest` walks the network and returns peer IDs from every queried peer's routing tables. In a long-lived test mesh, those tables may contain stale (disconnected) peer IDs that XOR-close to a topic. Publishes to those ghosts fall back to routed delivery and silently time out.

The reference `axona-peer` overrides `dht.findKClosest` in `browser_engine.js` to compute the K-closest set from local synaptome + self only. This guarantees every returned peer is reachable. The trade-off: in a huge mesh you'd want network-wide discovery, but for any deployment you can fit in your local synaptome (i.e., up to a few hundred peers), the local-only filter is more reliable.

If your peer talks to a long-lived test mesh, copy the override from `axona-peer/src/browser_engine.js`.

13.5 Replay needs the publish to still be cached

`since: 'all'` returns whatever's in the K-closest axons' replay caches right now. If you publish, the publisher's tab closes, and 60 seconds elapse with no subscribers, the empty role gets swept and the cache vanishes.

For durable history you need either:

- A "store" peer that subscribes to every topic and never leaves (acts as a passive archive), OR
- An application-layer journal (a database that mirrors every envelope).

13.6 Default replay window

Default replay-cache size is **100 messages per topic**. For chat rooms with high volume, you'll lose history past the last 100 messages. Increase it on busy topics by configuring the AxonaManager construction in your transport / engine wrapper, or paginate at the application layer.

13.6b Message size limit (and the silent-drop trap)

A published message is capped at **256 KiB** (the cap rose from 64 KiB in kernel v2.8.1), measured on the *serialized envelope*. Two things to know:

- **`peer.pub` rejects oversize up front** (kernel v2.8.2): you get `PublishError(PUBLISH_PAYLOAD_TOO_LARGE)` immediately, so catch it and handle large content differently rather than letting a message silently fail. (The root axon also enforces the cap at ingress as defense-in-depth.) A non-serializable message — circular ref, `BigInt` — throws `PUBLISH_INVALID_MESSAGE` instead.
- **Don't ship blobs over pub/sub even under the cap.** Every publish is replicated to K root axons, cached in their replay rings, and fanned to *all* subscribers — so a large message multiplies across the whole delivery tree. For images/documents, publish a small **content reference** (content hash + size + mime) and transfer the bytes out-of-band, content-addressed, fetched only by peers that want them.

13.7 Cross-version wire incompatibility

The bridge enforces a `MIN_PEER_VERSION` gate. Pre-1.1.0 peers send 16-character nodeIds; current peers send 66-character nodeIds. A peer too old gets close-code `4426` (UPGRADE_REQUIRED).

When you cut a wire-incompatible change, bump:

1. `KERNEL_VERSION` in `@axona/protocol/transport/handshake.js`
2. Your peer's `PEER_VERSION`
3. The bridge's `MIN_PEER_VERSION`

… and deploy bridge first, then peer.

13.8 Hard-reload after deploy

Browser bundle cache is sticky. After deploying a new peer, ask users (or yourself) to do a hard-reload (Cmd+Shift+R / Ctrl+Shift+R). A normal reload can serve the previous bundle from disk cache. If several tabs run different versions and the wire format moved, the mesh half-works and debugging is miserable.

---

14. Production checklist

Before launching a public Axona app:

- ☐ **TLS on the bridge** — nginx + certbot in front of the Node service. `axona-bridge/deploy/` has the exact recipe.
- ☐ **TURN credentials** — set `TURN_AUTH_SECRET` on the bridge and configure a TURN server (coturn) for NAT traversal in tough networks.
- ☐ **Stable bridge identity** — `BRIDGE_IDENTITY_PATH` pointing to a persistent disk location so the bridge's 264-bit nodeId survives restarts.
- ☐ **MIN_PEER_VERSION gate** — enforce so old peers get `UPGRADE_REQUIRED` rather than silently breaking.
- ☐ **Per-app identity persistence** — `dumpIdentity` / `loadIdentity` so your users keep their nodeId across sessions.
- ☐ **Signature verification policy** — decide which envelopes you trust by `signerPubkey` and reject the rest in your subscribe handler.
- ☐ **Replay-cache sizing** — bump from the default 100 if your topics are high-volume.

- ☐ **Health monitoring** — scrape `https://your-bridge/healthz` and/or hook `peer.health()` into your dashboards.
- ☐ **Topic naming policy** — public vs region-keyed vs publisher-keyed decided up front, documented, and consistent across pub/sub call sites.

---

15. API reference cheat sheet

Imports throughout this section assume `import { ... } from '@axona/protocol'`. Type annotations are informal; the kernel ships no TypeScript yet.

15.1 Identity

```
deriveIdentity({ lat, lng, extractable? })
  → Promise<Identity>

dumpIdentity(identity)
  → Promise<{ id, pubkey, privkey: base64-PKCS#8, region, createdAt }>

loadIdentity(envelope)
  → Promise<Identity>

// type Identity = {
//   id:         '<66 hex>',
//   pubkey:     Uint8Array(32),
//   pubkeyHex:  '<64 hex>',
//   privateKey: CryptoKey,           // Web Crypto Ed25519
//   region:     { lat, lng },
//   createdAt:  <ms>,
// }
```

15.2 Peer construction

```
new AxonaPeer({
  domain,                           // AxonaDomain (shared mesh state)
  node,                             // NeuronNode (local routing state)
  identity,                         // Identity from §15.1
  persist: <PersistenceAdapter>, // optional
})

await peer.start();
await peer.stop();
await peer.join({ ... });             // production bootstrap helper
await peer.leave({ drain, notify, timeoutMs });
```

15.3 Pub/sub

```
peer.pub(name, message, { publisher?, sign? })
  → Promise<msgId>

peer.sub(name, handler, { publisher?, since? })
  → Promise<Subscription>   // handler also gets { deleted, msgId, topic } on a kill

peer.unsub(name, { publisher? })
  → Promise<{ ok, removed }>                        // leave a topic (v2.10.0)
```

```
peer.pull(msgId | null, { topic, publisher, timeoutMs })
  → Promise<Envelope | null>                          // null msgId → latest (v2.10.0)

peer.kill(name, msgId, { publisher? })
  → Promise<{ ok }>                                   // creator-only retract (v2.10.0)

peer.touch(name, msgId, { publisher? })
  → Promise<{ ok }>                                   // creator-only keep-alive (v2.16.0)

peer.unpub(name, { destroy?, publisher? })
  → Promise<{ ok }>                                   // owner-only queue removal (v2.10.0)

peer.metrics(topic, { publisher, timeoutMs })
  → Promise<{ publishes, subscribers, reshare_count }>

await subscription.stop();
```

Topic limits (v2.10.0): max 100 (256) messages / 24 h hold (48 h ceiling) / deterministic seq-ordered eviction / per-publisher ¼ quota on open topics.

15.4 Direct messaging

```
peer.send(peerIdHex, message)           // request/response
  → Promise<reply>                       // reply = whatever the remote handler returned

peer.notify(peerIdHex, message)         // fire-and-forget
  → Promise<boolean>                     // true if enqueued; return value discarded

peer.onMessage(handler)                 // single handler, replaces previous
  // handler signature: (senderIdHex, message) => reply | Promise<reply>
```

15.5 Mesh introspection

```
peer.peers()                            // → string[] of 66-char hex
peer.onPeerJoin(handler)                // handler(peerIdHex, ctx) → returns unsubscribe()
peer.onPeerLeave(handler)               // handler(peerIdHex, ctx) → returns unsubscribe()
peer.lookup(targetBigInt)               // → { found, hops, time, path }
peer.health()                           // → diagnostic snapshot
peer.onLog(handler)
peer.onError(handler)
```

15.6 Topic ID derivation

```
deriveTopicId(publisherIdOrNull, name)
  → Promise<'<66 hex>'>

// Conventions:
// - publisher = null          → public mode, '00' prefix
// - publisher = '<66 hex>'    → publisher-keyed; uses publisher's S2 prefix
// - publisher = synthRegionId → region-keyed; uses region's S2 prefix
```

15.7 Envelope helpers

```
buildEnvelope({ topic, message, ts?, seq?, identity, sign? })
  → Promise<Envelope>                  // env carries seq (format v2)
```

```
verifyEnvelope(envelope)
  → Promise<{ ok, reason?, signed }>   // check .ok, NOT truthiness; needs seq

checkFreshness(envelope, { now?, maxSkewMs? })
  → { ok, reason? }                    // C-2 window; ingress-only, not in verify

computeMsgId({ seq, topic, ts, message, signature? })
  → Promise<'<64 hex>'>
```

15.8 Geo helpers

```
geoCellId(lat, lng, bits)               // → integer S2 cell ID
haversine(lat1, lng1, lat2, lng2)       // → km
roundTripLatency(lat1, lng1, lat2, lng2) // → ms (rough)
```

15.9 Hex/ID math

```
ID_BITS                                 // 264
HEX_CHARS                               // 66
isHexId(s)                              // bool
toHex(bigint)                           // 66-char hex
fromHex(hex)                            // BigInt
s2PrefixOfHex(hex)                      // int 0..255
extractHash(hex)                        // bottom 256 bits as hex
xorDistance(aBigInt, bBigInt)           // BigInt
```

15.10 Persistence adapters

```
import { IndexedDBPersistence } from '@axona/protocol/persistence/indexeddb.js';
import { FilePersistence }      from '@axona/protocol/persistence/file.js';

const adapter = new IndexedDBPersistence({ dbName: 'my-app' });
// or
const adapter = new FilePersistence({ dir: './state' });   // JSON files under dir (default ./.axona)
```

15.11 Errors

All thrown errors are subclasses of `AxonaError` with a stable `.code`:

Class	Codes
`IdentityError`	`IDENTITY_INVALID`, `IDENTITY_LOAD_FAILED`, …
`TransportError`	`TRANSPORT_CLOSED`, `TRANSPORT_NOT_BOUND`, `TRANSPORT_TIMEOUT`, …
`PublishError`	`PUBLISH_INVALID_TOPIC`, `PUBLISH_SIGN_FAILED`, `PUBLISH_PAYLOAD_TOO_LARGE`, …
`SubscribeError`	`SUBSCRIBE_INVALID_TOPIC`, `SUBSCRIBE_HANDLER_MISSING`, …
`KillError`	`KILL_INVALID_TOPIC`, `KILL_INVALID_MSGID`, `KILL_SIGN_FAILED`
`UnpubError`	`UNPUB_INVALID_TOPIC`, `UNPUB_PUBLIC_TOPIC`, `UNPUB_SIGN_FAILED`
`PullError`	`PULL_INVALID_MSGID`, `PULL_AXONS_UNREACHABLE`
`MetricsError`	`METRICS_AXONS_UNREACHABLE`
`UpgradeRequiredError`	`UPGRADE_REQUIRED` (the 4426 close)

Class	Codes

Switch on `err.code` rather than `err.message` when handling programmatically.

15.12 Version constants

```
WIRE_VERSION          // '1.0'
KERNEL_VERSION        // '2.32.0'
UPGRADE_CLOSE_CODE    // 4426
```

Where to go next

- **Quick Start** — if someone you're onboarding has five minutes, send them here instead of this 1500-line guide.
- **API Reference** — when you're past the conceptual material and just need the signature for a specific call.
- **Read the source of the reference peer**: `axona-peer/src/client.js` is the canonical end-to-end consumer. ~1500 lines, well-commented, builds a full browser pub/sub UI on top of the kernel.
- **Read the architecture spec**: `architecture/N-DHT-Architecture.md` for the formal protocol contracts (Transport, DHT, BootstrapService) and routing math. Not required for app development; required for understanding *why* things are the way they are.
- **Read the wire protocol**: `implementation/Axona-Wire-Protocol-v0.71.md` if you need to implement Axona in another language, or wire it into a non-WebSocket transport.
- **Run the simulator**: `dht-sim` renders an Axona mesh on a 3D globe and lets you tune routing parameters interactively. Great for building intuition.

Found a bug or a gap in this guide? Open an issue at https://github.com/axona-net/axona-docs.